

ROYAL SQUARE FINANCIAL

Client & Advisor Platform

TECHNOLOGY STACK

AfriHack 2026

Document	Technology Stack Specification
Project	Royal Square Financial Platform
Version	1.0
Date	19 September 2026

This document defines the agreed technology choices for the AfriHack 2026 prototype.

1. Stack Overview

Layer	Technology	Purpose
Frontend	React	Client App + Advisor Portal
Backend	Python + FastAPI	REST APIs, business logic, RAG, integrations
Database	Supabase (PostgreSQL)	All structured data
File Storage	Supabase Storage	Claim photos, documents, sketches
Auth	Supabase Auth	Client & Advisor authentication
Frontend Hosting	Vercel	React application deployment
Backend Hosting	Render	FastAPI service deployment
LLM / Chat	Groq (recommended) or Gemini	RAG chatbot + email assistance
Vector Store	Supabase pgvector or Chroma	Document embeddings for RAG

2. Frontend — React

React is used for both the Client-facing application and the Advisor Portal. A single codebase with role-based routing is preferred to reduce duplication.

Recommended supporting libraries:

- React Router — navigation and protected routes
- Tailwind CSS or similar utility CSS — fast, consistent UI
- Axios or React Query / TanStack Query — API communication and caching
- React Hook Form — forms for claims and requests
- Lucide React or Heroicons — icons

Deployment target: **Vercel** (excellent React support, preview deployments, free tier suitable for hackathon).

3. Backend — Python + FastAPI

FastAPI provides high performance, automatic OpenAPI documentation, and excellent developer experience. It will expose all business APIs used by both React frontends and will host the RAG and email-related logic.

Key FastAPI capabilities used:

- REST endpoints for clients, claims, goals, reminders, requests
- File upload endpoints for claim documents and photos
- RAG query endpoint for the document assistant
- Google OAuth / Gmail-related endpoints (for the email integration)
- Background tasks where useful (e.g. reminder checks)

Deployment target: **Render** (simple Python web service deployment, free tier available).

4. Database & Storage — Supabase

Supabase is used as the primary data platform. It provides PostgreSQL, authentication, and object storage under one project, which keeps the hackathon setup simple and fast.

- **PostgreSQL** — clients, policies, goals, reminders, claims, claim updates, requests, users
- **Supabase Auth** — email/password or magic-link login for Clients and Advisors, with role claims
- **Supabase Storage** — secure storage of photos, licences, sketches and other uploaded documents
- **pgvector** (optional) — can be enabled for storing document embeddings used by the RAG system

5. RAG Document Assistant & LLM Choice

The Advisor-side chatbot uses Retrieval-Augmented Generation. Only approved documents (policy wordings, internal processes, regulations, company policies) are indexed. Answers must include citations (document name + page number).

Recommended free / low-cost LLM options:

Provider	Notes	Recommendation
Groq	Very fast inference, generous free tier, Llama and Mixtral models	Primary recommendation
Google Gemini	Strong free tier, good quality, easy API	Strong alternative
Together.ai	Good free credits, many open models	Backup option
OpenRouter	Access to many models with free credits	Flexible backup

Grok API is not recommended as the primary free option for a third-party hackathon app at this time (access and rate limits are more constrained for external use). Groq or Gemini are more practical for a free-tier prototype.

Supporting libraries: LangChain or LlamaIndex for the RAG pipeline; embeddings from the same provider or a free embedding model.

6. Google Workspace / Email Integration

The platform will integrate with Google Workspace (Gmail focus) so advisers can surface important emails, retrieve thread context, flag messages and receive help drafting replies. This is implemented via Google OAuth 2.0 and the Gmail API.

Required pieces:

- Google Cloud project with Gmail API enabled
- OAuth consent screen and credentials
- FastAPI endpoints to handle OAuth flow and token storage
- Scoped access (read / send as needed) with clear user consent

For AfriHack a simulated or limited demo of this capability is acceptable if full live OAuth proves too time-consuming. Full production integration remains a priority after the hackathon.

7. Deployment

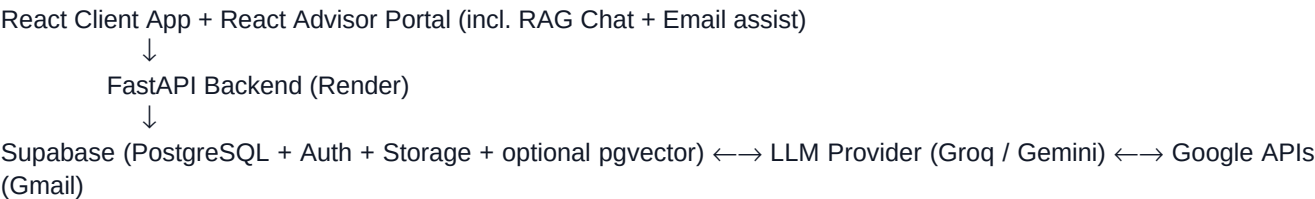
- **Frontend (React)** → Vercel
- **Backend (FastAPI)** → Render
- **Database & Storage** → Supabase (managed)

Environment variables will be used for all secrets (Supabase keys, LLM API keys, Google OAuth credentials).

8. Supporting Tools & Libraries

- Python packages: fastapi, uvicorn, sqlalchemy or supabase-py, python-multipart, pydantic, langchain / llama-index, httpx
- Frontend: Vite + React, Tailwind CSS, React Router, TanStack Query
- Version control: Git + GitHub
- API testing: FastAPI automatic Swagger UI + optional Postman / Thunder Client

9. High-Level Architecture



10. Important Notes for the Team

- Keep the AfriHack demo focused on the core claims flow, dashboards, goals/reminders and a working RAG example.
- Email integration can be shown as a working prototype or as a clear next-step roadmap item if time is short.

- Use demo/seed data only — do not connect to Royal Square's live production systems during the hackathon.
- Store all secrets in environment variables; never commit keys to Git.

End of Technology Stack Document
Royal Square Financial — AfriHack 2026